

Basic concept of an exception, the Exception class, try-catch, throw, throws and finally.

06 October 2026 05:31

Exception Handling in Java

Exception handling in Java is one of the most powerful mechanism to handle runtime errors so that the normal flow of application can be maintained.

What is exception in Java?

In Java, an exception is an event that occurs during the execution of a program and disrupts the normal flow of instruction. Exception can occur for various reason such as invalid user input of file not being found or division by 0. When an exception occurs, it is typically represented by an object of a subclass of the `java.lang.Exception` class.

What is Exception Handling in Java?

Exception handling is a mechanism that allows a program to handle runtime errors such as `ClassNotFoundException`, `IOException`, `SQLException`, `RemoteException` and others so that the normal flow of the application can be maintained.

Hierarchy of Java Exception classes

Exceptions are represented by classes and all the exception classes are the part of `java.lang` package. They are organized in a hierarchical structure with **Throwable** at the top. The main hierarchy is as follows:

- **Throwable:** The super class of all errors and exceptions in Java.
 - **Error:** Represents serious system errors that application usually cannot handle. (example, `OutOfMemoryError`)
 - **Exception:** Represents conditions that a program might want to catch.
 - **CheckedException:** Must be either handled using `try - catch` or declared using `throws`(e.g., `IOException`, `SQLException`).
 - **UncheckedException(RuntimeException):** Do not need explicit handling. Typically caused by programming errors (example, `NullPointerException`, `ArithmeticException`.)

The following diagram demonstrates the same:

App.java

Input two numbers:

10 0

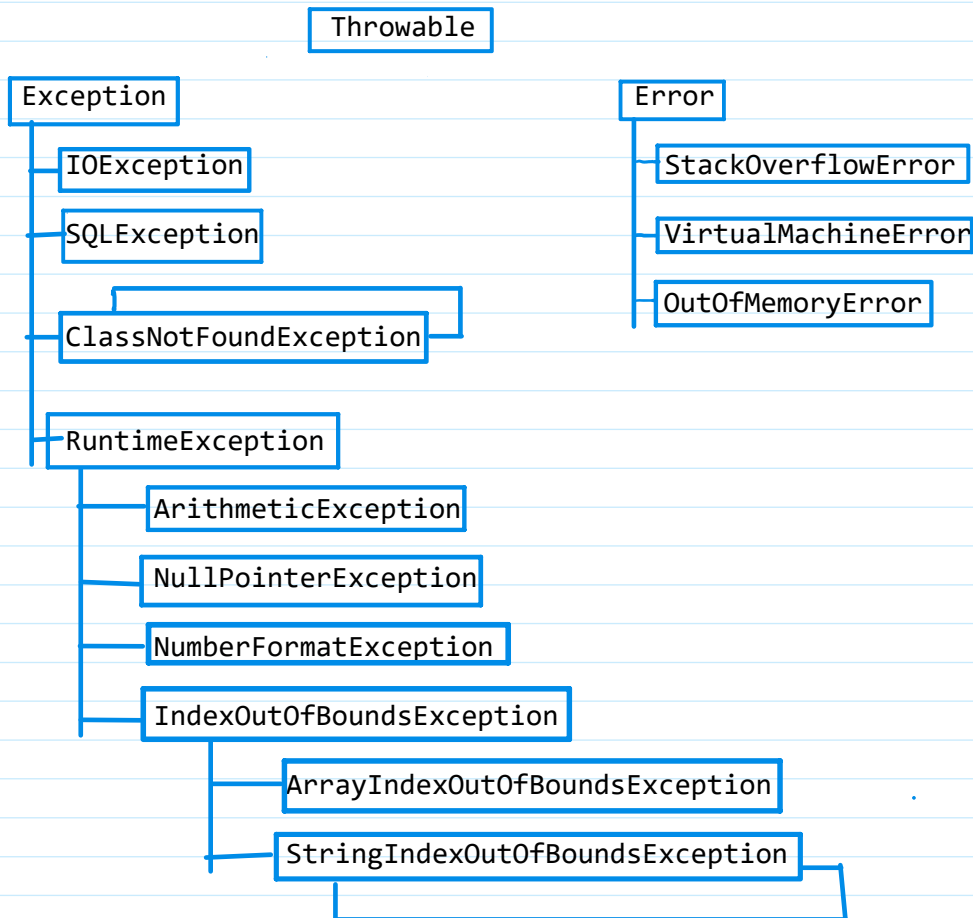
0

division by 0 is not allowed, there is an exception, the code will halt or terminate on that point by displaying exception.

Throwable

Exception

Error



Java Checked Exceptions

Checked exceptions are the exceptions that are checked at compile - time. This means that compiler verifies that code handles these exceptions either by catching them or declaring them in the method signature using the throws keyword. Examples of the checked exceptions include:

- **IOException:** An exception is thrown when an input / output operation fails, such as when reading from or writing to a file.
- **SQLException:** It is thrown when an error occurs while accessing a database.
- **ParseException:** Indicates a problem while parsing a string into another data type, such as parsing a date.
- **ClassNotFoundException:** It is thrown when an application tries to load a class through its string name using methods like `Class.forName()`, but the class with the specified name cannot be found in the class path.

```

11 public class CheckedExceptionExample
12 {
13     public static void main(String[] args){
14         File file = new File("example.txt");
15         FileReader fr = new FileReader(file);
16     }
17 }
  
```

The possible exception of type `java.io.FileNotFoundException` needs to be handled

We need to use either

- try - catch
- throws with method signature

```
public static void main(String[] args){
    try{
        File file = new File("example.txt");
        FileReader fr = new FileReader(file);
        fr.close();
    }
    catch {
    }
}
```

The possible exception of type java.io.IOException needs to be handled

- Fix: Surround this line with try/catch
- Fix: Add throws statement for this exception in the containing method

Checked exceptions are detected by Java compiler

Messages/suggestions are generated by Java compiler.

```
package ExceptionHandling;
import java.io.*;
```

```
/**
 * Write a description of class CheckedExceptionExample here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
```

```
public class CheckedExceptionExample
{
```

```
    public static void main(String[] args){
        try{
```

```
            File file = new File("example.txt");
            FileReader fr = new FileReader(file);
            fr.close();
        }
```

may throw an IOException, which is a checked exception.

```
        catch(FileNotFoundException ob){
            System.out.print("\nFile nahi mila");
        }
```

Suppressing the JVM message with our message, that is user friendly.

```
        catch(IOException ob){
            System.out.print("\nFile read error");
        }
```

```
    }
```

```
}
```

Output:

```
Blue: Terminal Window - Krittin11
Options
File nahi mila
```

Java Unchecked Exception(Runtime Exceptions)

Unchecked exception, also known as runtime exceptions, are not checked at compile - time. These exceptions usually occur due to programming errors such as logic errors or incorrect assumption in the code. They do not need to be declared in the method signature using the throws keyword, making it optional to handle them. Examples of unchecked exception include:

- **NullPointerException:** It is thrown when trying to access or call a method on an object reference that is null.
- **ArrayIndexOutOfBoundsException:** It occurs when we try to access an array

element with an invalid index.

- **ArithmeticException:** It is thrown when an arithmetic operation fails, such as division by 0.
- **IllegalArgumentException:** It indicates that the method has been passed an illegal or inappropriate argument.

```
public class UncheckedExceptionExample
{
    public static void main(String[] args){
        String str = null;
        //String str = ""; is not null
        System.out.print("\nLength of String str: " + str.length());
    }
}
```

Exception class name → `java.lang.NullPointerException`: Cannot invoke "String.length()" because "str" is null
message → `at ExceptionHandling.UncheckedExceptionExample.main(UncheckedExceptionExample.java:15)`

```
21 int A[] = {1,2,3};
22 System.out.print("\nA[5]: " + A[5]);
23 }
24 }
```

```
java.lang.ArrayIndexOutOfBoundsException:
Index 5 out of bounds for length 3
```

```
String value is null.
java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 3
at ExceptionHandling.UncheckedExceptionExample.main(UncheckedExceptionExample.java:22)
```

```
package ExceptionHandling;
```

```
/**
 * Write a description of class UncheckedExceptionExample here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
```

```
public class UncheckedExceptionExample
{
    public static void main(String[] args){
        String str = null;
        //String str = ""; is not null
        try{
            System.out.println("\nLength of String str: " + str.length());
        }
        catch(NullPointerException npe){
            System.err.println("\nString value is null.");
        }

        int A[] = {1,2,3};
        try{
```

```

        System.out.print("\nA[5]: " + A[5]);
    }
    catch(ArrayIndexOutOfBoundsException aiob){
        System.err.println("\nArray index is invalid");
    }
}
}

```

Output:

```

String value is null.

Array index is invalid

```

```

package ExceptionHandling;
import java.util.*;

/**
 * Write a description of class UncheckedExceptionExample here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class UncheckedExceptionExample
{
    public static void main(String[] args){
        String str = null;
        //String str = ""; is not null
        try{
            System.out.println("\nLength of String str: " + str.length());
        }
        catch(NullPointerException npe){
            System.err.println("\nString value is null.");
        }

        int A[] = {1,2,3};
        try{
            System.out.print("\nA[5]: " + A[5]);
        }
        catch(ArrayIndexOutOfBoundsException aiob){
            System.err.println("\nArray index is invalid");
        }

        int r, t, s;
        System.out.print("\nEnter two numbers: ");
        Scanner sc = new Scanner(System.in);
        t = sc.nextInt();
        s = sc.nextInt();
        r = t/s;
        System.out.print("\nQuotient: " + r);
    }
}

```

Output:

Enter two numbers: 20

0

Can only enter input while your program is running

String value is null.

Array index is invalid

java.lang.ArithmeticException: / by zero

at ExceptionHandling.UncheckedExceptionExample.main(UncheckedExceptionExample.java:35)

```
package ExceptionHandling;
import java.util.*;
```

```
/**
 * Write a description of class UncheckedExceptionExample here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class UncheckedExceptionExample
{
    public static void main(String[] args){
        String str = null;
        //String str = ""; is not null
        try{
            System.out.println("\nLength of String str: " + str.length());
        }
        catch(NullPointerException npe){
            System.err.println("\nString value is null.");
        }

        int A[] = {1,2,3};
        try{
            System.out.print("\nA[5]: " + A[5]);
        }
        catch(ArrayIndexOutOfBoundsException aiob){
            System.err.println("\nArray index is invalid");
        }

        int r, t, s;
        System.out.print("\nEnter two numbers: ");
        Scanner sc = new Scanner(System.in);
        t = sc.nextInt();
        s = sc.nextInt();
        try{
            r = t/s;
            System.out.print("\nQuotient: " + r);
        }
        catch(ArithmeticException ae){
            System.err.println("\nDivisor cannot be 0");
        }
    }
}
```

Output:

```
Enter two numbers: 10
0
```

Can only enter input while your program is running

String value is null.

Array index is invalid

Divisor cannot be 0

Exceptions handled

code continued and Exited Gracefully.

```
33 Scanner sc = new Scanner(System.in);
34 t = sc.nextInt();
35 s = sc.nextInt();
36 try{
37     r = t/s;
38     System.out.print("\nQuotient: " + r);
39 }
40 catch(ArithmeticException ae){
41     System.err.println("\nDivisor cannot be 0");
42 }
43 }
```

java.util.InputMismatchException:
null (in java.util.Scanner)

```
Enter two numbers: ten
```

Can only enter input while your program is running

String value is null.

Array index is invalid

java.util.InputMismatchException

at ExceptionHandling.UncheckedExceptionExample.main(UncheckedExceptionExample.java:33)

```
package ExceptionHandling;
import java.util.*;
```

```
/**
```

```
 * Write a description of class UncheckedExceptionExample here.
```

```
 *
```

```
 * @author (your name)
```

```
 * @version (a version number or a date)
```

```
 */
```

```
public class UncheckedExceptionExample
{
```

```
    public static void main(String[] args){
```

```
        String str = null;
```

```
        //String str = ""; is not null
```

```
        try{
```

```
            System.out.println("\nLength of String str: " + str.length());
```

```

    }
    catch(NullPointerException npe){
        System.err.println("\nString value is null.");
    }

    int A[] = {1,2,3};
    try{
        System.out.print("\nA[5]: " + A[5]);
    }
    catch(ArrayIndexOutOfBoundsException aiob){
        System.err.println("\nArray index is invalid");
    }

    int r, t, s;
    System.out.print("\nEnter two numbers: ");
    Scanner sc = new Scanner(System.in);
    try{
        t = sc.nextInt();
        s = sc.nextInt();
        try{
            r = t/s;
            System.out.print("\nQuotient: " + r);
        }
        catch(ArithmeticException ae){
            System.err.println("\nDivisor cannot be 0");
        }
    }
    catch(InputMismatchException e){
        System.err.println("\nPlease enter integer value.");
    }
}
}

```

Output:

```

Enter two numbers: 10
two

Can only enter input while you

String value is null.

Array index is invalid

Please enter integer value.

```

Java Errors

Errors represent exceptional conditions that are not expected to be caught under normal circumstances. They are typically caused by issues outside the control of the application, such as system failures or resource exhaustion. Errors are not meant to be caught or handled by application code. Examples of errors include:

- **OutOfMemoryError:** It occurs when the Java Virtual Machine (JVM) cannot allocate enough memory for the application.

- **StackOverflowError:** It is thrown when the stack memory is exhausted due to excessive recursion.
- **NoClassDefFoundError:** It indicates that JVM cannot find the definition of a class that was available at compile-time.